

Arrays – Zugriff und Iteration

Pseudocode (IHK-Stil):

```
// Pseudocode: Array anlegen und durchlaufen
A = neues Array[5]
A[0] = 10
A[1] = 20
A[2] = 30
for i = 0 to 4
    ausgabe(A[i])
end for
```

Java-Beispiel:

```
public class ArrayBeispiel {
    public static void main(String[] args) {
        int[] A = new int[5];
        A[0] = 10;
        A[1] = 20;
        A[2] = 30;
        for (int i = 0; i < A.length; i++) {
            System.out.println(A[i]);
        }
    }
}
```

Arrays – Erklärung & Beispiele

Ein Array ist eine feste Datenstruktur mit ****fester Länge****. Die Elemente werden mit einem Index (0..n-1) angesprochen.

Zugriff auf ein Element: $O(1)$ (direkter Speicherzugriff).

****Typische Fehlerquellen:****

- `ArrayIndexOutOfBoundsException`: Zugriff außerhalb der Grenzen
- Initialisierung vergessen: Standardwerte (0, null)
- Länge ist fest: Erweiterung nicht möglich

****Prüfungstipp:**** Array-Größe korrekt angeben und Schleifen-Bedingung mit `i < A.length` schreiben, nicht `i <=`.

Index	Wert
0	10
1	20
2	30
3	0 (default)
4	0 (default)

Listen (ArrayList) – Hinzufügen, Entfernen, Iteration

Pseudocode (IHK-Stil):

```
liste = neue Liste<String>
liste.add("Apfel")
liste.add("Birne")
liste.add("Banane")
for i = 0 to liste.size()-1
    ausgabe(liste.get(i))
end for
liste.remove("Birne")
```

Java-Beispiel:

```
import java.util.ArrayList;

public class ListenBeispiel {
    public static void main(String[] args) {
        ArrayList<String> liste = new ArrayList<>();
        liste.add("Apfel");
        liste.add("Birne");
        liste.add("Banane");
        for (String s : liste) {
            System.out.println(s);
        }
        liste.remove("Birne");
    }
}
```

Listen – Erklärung & Beispiele

Eine Liste ist eine dynamische Datenstruktur (variable Länge). In Java z. B. `ArrayList` oder `LinkedList`.

****Unterschied Array vs. ArrayList:****

- Array: feste Länge, schneller Zugriff ($O(1)$)
- ArrayList: wächst dynamisch, Einfügen am Ende $O(1)$, in der Mitte $O(n)$
- LinkedList: schnelles Einfügen/Löschen $O(1)$, aber Zugriff auf Index $O(n)$

****Typische Fehlerquellen:****

- Schleifenänderung während Iteration (`ConcurrentModificationException`)
- Null-Werte in der Liste
- Unterschied `remove(index)` vs. `remove(Object)` in Java

****Prüfungstipp:**** Listen werden in Prüfungen oft für dynamische Datensätze genutzt, z. B. Kundenlisten.

Operation	Ergebnis
<code>add(Apfel, Birne, Banane)</code>	<code>[Apfel, Birne, Banane]</code>
<code>remove(Birne)</code>	<code>[Apfel, Banane]</code>
<code>size()</code>	2

Maps (Schlüssel-Wert) – put, get, remove

Pseudocode (IHK-Stil):

```
map = neue Map<Integer, String>
map.put(1, "Apfel")
map.put(2, "Birne")
map.put(3, "Banane")
wert = map.get(2) // Birne
map.remove(3)
```

Java-Beispiel:

```
import java.util.HashMap;

public class MapBeispiel {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(1, "Apfel");
        map.put(2, "Birne");
        map.put(3, "Banane");
        System.out.println(map.get(2)); // Birne
        map.remove(3);
    }
}
```

Maps – Erklärung & Beispiele

Eine Map speichert Schlüssel-Wert-Paare. Jeder Schlüssel ist eindeutig, Werte dürfen mehrfach vorkommen.

In Java: `HashMap<K,V>`, `TreeMap<K,V>`.

Operationen:

- `put(key, value)`: Einfügen
- `get(key)`: Wert zum Schlüssel ($O(1)$ bei `HashMap`)
- `remove(key)`: Löschen
- `containsKey` / `containsValue`: Suche

Typische Fehlerquellen:

- Zugriff auf nicht existierenden Schlüssel $\rightarrow$ null
- `HashMap` ungeordnet $\rightarrow$ Reihenfolge nicht garantiert
- Null als Schlüssel nicht überall erlaubt

Prüfungstipp: Maps eignen sich für Assoziationen (z. B. Kundennummer $\rightarrow$ Kunde).

Schlüssel	Wert
1	Apfel
2	Birne
3	Banane